B-drzewa

1. Wprowadzenie

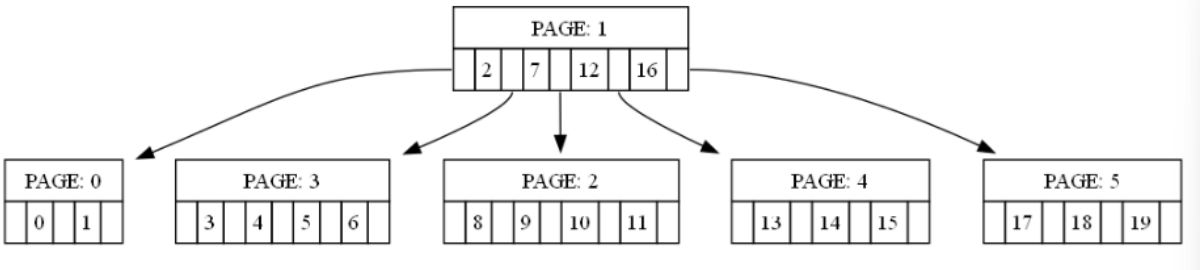
B-drzewo jest to drzewiasta, zbilansowana struktura danych służąca do przechowywania kluczy w pewnym porządku oraz powiązanych z nimi danych. Wyobraźmy sobie plik, w którym mamy zapisane rekordy, które są identyfikowalne kluczami, i teraz mielibyśmy na tym pliku obsługiwać takie operacje jak wyszukiwanie danych po kluczu. Bez żadnej dodatkowej struktury, wyszukanie tego rekordu będzie w najgorszym wypadku wymagało $\frac{N}{2b}$ operacji, gdzie N oznacza liczbę rekordów, a b współczynnik blokowania. Wraz ze wzrostem wielkości plików taka ilość operacji dyskowych staje się nieakceptowalna, ponieważ w najgorszym wypadku trzeba wczytać i przeszukać cały plik. Jednym ze sposobów rozwiązania tego problemu jest wykorzystanie plików indeksowych, a dokładniej w naszym przypadku, B-drzew.

2. Opis struktury

2.1. Różnice względem innych klasycznych struktur drzewiastych

B-drzewo jest drzewiasta struktura danych, która dzięki stałemu bilansowaniu utrzymuje złożoność logarytmiczną $O(\log n)$ dla wszystkich podstawowych operacji, takich jak wyszukiwanie, wstawianie i usuwanie.

Różni się od innych popularnych zbilansowanych drzew binarnych tym, że pojedynczy wierzchołek drzewa może posiadać wiele kluczy. Wynika to z faktu, że w związku z tym, że przy kosztownych odczytach z dysku występuje zjawisko odczytu blokowego, więc chcemy zmaksymalizować ilość informacji odebranej przy pojedynczej operacji. W związku z tym, nasze drzewo nie posiada odnośników do dwójki dzieci (mniejszego i większego niż rodzic), lecz jeżeli mamy k kluczy, to posiadamy $k + 1$ odnośników do dzieci.



Dodatkowo B-drzewo nie przechowuje zazwyczaj samych danych, lecz pełni funkcję indeksu, zwierając informacje o tym, gdzie dokładnie w głównym pliku z danymi są zapisane rekordy. Oznacza to, że kiedy już za pomocą B-drzewa wyszukamy lokalizację rekordu, do jego odczytu potrzebna będzie maksymalnie jedna dodatkowa operacja (lub zero, jeśli blok danych już znajdował się w pamięci operacyjnej).

Często, aby usprawnić algorytmy, które się wykonuje na B-drzewach, dodaje się również inne informacje w każdym wierzchołku, takie jak np. wskaźnik do rodzica.

2.2. Zasady B-drzew

B-drzewa posiadają 4 główne zasady:

Niech $d > 0$

1. Wszystkie liście są na tej samej wysokości,
2. Każdy wierzchołek zawiera co najwyżej $2d$ kluczy,
3. Każdy wierzchołek zawiera co najmniej d kluczy, z wyjątkiem korzenia, który może zawierać 1 klucz,
4. Jeżeli wierzchołek nie jest liściem i ma k kluczy, to posiada $k + 1$ dzieci.

Pierwsza zasada zapewnia stałe równoważenie drzewa, co jest kluczowe dla zachowania logarytmicznej złożoności. Druga i trzecia zapewniają minimum 50-procentowy stopień zapełnienia drzewa (poza korzeniem), co gwarantuje efektywne wykorzystanie miejsca na dysku.

2.3. Parametry b-drzew

Bardzo łatwo wywnioskować na podstawie opisu tej struktury, iż szybkość jej działania jest bezpośrednio związana z wysokością drzewa (h). Bardzo łatwo również zauważyć, że wysokość drzewa jest bezpośrednio związana z wartością zmiennej d. Im większe będzie nasze d, tym nasze drzewo na ogół będzie niższe, co oznacza mniejszą ilość operacji dyskowych. Z tego właśnie powodu, powinno się maksymalizować tę zmienną tak, by wielkość danych wierzchołka mieściła się wciąż na jednej stronie dyskowej.

Ze względu na to, że nie wszystkie wierzchołki będą przechowywały tej samej ilości kluczy, to nie da się zazwyczaj stwierdzić na podstawie ilości rekordów, jaka jest dokładna wysokość drzewa. Możemy za to określić, w jakim przedziale na pewno będzie się znajdować wysokość drzewa.

Zastanówmy się, jaka możemy być minimalna ($\boldsymbol{N_{min}}$) i maksymalna ($\boldsymbol{N_{max}}$) ilość kluczy w drzewie o wysokości h i danej wartości d.

Minimalna ilość kluczy występuje wtedy, gdy korzeń ma 1 klucz, a każdy inny wierzchołek ma ich d.

$$1 + 2d + 2d(d+1) + \cdots + 2d(d+1)^{h-2} =$$
$$1 + 2d\frac{1-(d+1)^{h-1}}{1-(d+1)} = \boldsymbol{1 + 2\left((d+1)^{h-1} - 1\right) = N_{min}}$$

Maksymalna ilość kluczy z kolei występuje wtedy, gdy każdy wierzchołek ma $2d$ kluczy.

$$2d + 2d(2d+1) + 2d(2d+1)^2 + \cdots + 2d(2d+1)^{h-1} =$$
$$2d\frac{1-(2d+1)^h}{1-(2d+1)} = \boldsymbol{(2d+1)^h - 1 = N_{max}}$$

Zauważmy, że wartość $\boldsymbol{N_{min}}$ odpowiada najwyższemu drzewu z N kluczami, a $\boldsymbol{N_{max}}$ najniższemu drzewu z N kluczami. Wyznaczmy teraz wysokości drzew względem tych wartości:

$$\mathbf{1} + \mathbf{2}\left((\boldsymbol{d}+\mathbf{1})^{\boldsymbol{h}-\mathbf{1}} - \mathbf{1}\right) = \boldsymbol{N_{min}} \quad \rightarrow \quad h = \mathbf{log}_{\boldsymbol{d}+\mathbf{1}}\left(\frac{\boldsymbol{N_{min}}+\mathbf{1}}{\mathbf{2}}\right) + \mathbf{1}$$

$$(\mathbf{2}\boldsymbol{d}+\mathbf{1})^{\boldsymbol{h}} - \mathbf{1} = \boldsymbol{N_{max}} \quad \rightarrow \quad h = \mathbf{log}_{\mathbf{2}\boldsymbol{d}+\mathbf{1}}(\boldsymbol{N_{max}}+\mathbf{1})$$

Możemy, więc napisać, że dla drzewa o N kluczach, jego wysokość h spełnia nierówność:

$$\lceil \mathbf{log}_{\mathbf{2}\boldsymbol{d}+\mathbf{1}}(\boldsymbol{N}+\mathbf{1}) \rceil \leq h \leq \left\lfloor \mathbf{log}_{\boldsymbol{d}+\mathbf{1}}\left(\frac{\boldsymbol{N}+\mathbf{1}}{\mathbf{2}}\right) + \mathbf{1} \right\rfloor$$

Jeżeli chodzi o wykorzystanie pamięci to łatwo zauważyć, że dzięki zasadom 1 i 2 mamy gwarantowane zapełnienie tych drzew kluczami minimalnie ok. 50%, a maksymalnie nawet 100%. Pytanie jest więc, jaka jest średnia wartość wykorzystania pamięci. Odpowiedź zależy od typu operacji wykonywanych na drzewie. Na ogół udowodniono, że przy losowych operacjach dodawania i usuwania otrzymujemy $\ln 2$, czyli około 70%.

2.4. Algorytmy

To, w jaki sposób algorytmy działają na B-drzewach, jest bardzo podobne z założenia do tego, jak to wygląda w innych samobilansujących się strukturach drzewiastych. Wykonujemy zadaną operację, a następnie sprawdzamy, czy wszystkie z naszych podstawowych zasad wciąż się zgadzają. Jeżeli są spełnione, to kończymy algorytm. Jeżeli nie, to naprawiamy naszą strukturę, zaczynając od dołu drzewa i propagując naprawy w górę, ponieważ operacje „naprawiania” mogą z kolei zniszczyć strukturę rodzica.

Mamy 3 główne sposoby „naprawiania” naszej struktury po takich operacjach:

- Kompensacja – można ją zastosować, gdy mamy niedobór kluczy lub nadmiar. Polega na przeniesieniu lub zabraniu kluczy od bezpośredniego rodzeństwa. Jest to często znacznie bardziej wydajna opcja, ponieważ nie wymaga od nas tworzenia ani usuwania stron dyskowych i nie może spowodować uszkodzenia struktury rodzica. Warunkiem jest posiadanie sąsiada, który może oddać lub przyjąć klucze.
- Rozszczepienie – stosujemy ją w przypadku, gdy mamy przepełnienie ($2d+1$ kluczy), a kompensacja nie jest możliwa. Polega na podzieleniu przepełnionego wierzchołka na dwa nowe, oba z d kluczami. Klucz środkowy awansuje do rodzica. Konsekwencją jest konieczność sprawdzenia, czy nie wystąpiło przepełnienie u rodzica.
- Scalenie – jest to odwrotność rozszczepienia, którą stosujemy w przypadku niedoboru kluczy (mniej niż d kluczy). Brak możliwości wykonania kompensacji oznacza, że bezpośrednie rodzeństwo ma d kluczy. W takiej sytuacji łączymy klucze z naszego wierzchołka, klucze sąsiada oraz jeden klucz od rodzica, otrzymując jeden nowy wierzchołek z $2d$ kluczami. Powoduje to usunięcie klucza z rodzica, co z kolei wymaga sprawdzenia, czy nie wystąpił w nim niedobór.

3. Porównanie praktyki z teorią

3.1. Implementacja

W celach przetestowania skuteczności B-drzewa w praktyce zaimplementowałem własne zasady buforowania w pamięci operacyjnej, aby jak najlepiej odwzorować działanie tej struktury w rzeczywistości. Do zarządzania buforem wykorzystałem algorytm LRU (Least Recently Used). Mówi on, że gdy koniecznie jest zwolnienie miejsca dla nowych danych, należy usunąć z pamięci podręcznej element, który był najdawniej używany. Aby w efektywny sposób kontrolować priorytety elementów bufora, użyłem listy dwukierunkowej, która nam pozwala na usuwanie, dodawanie oraz sprawdzanie ostatniego elementu w stałym czasie $O(1)$.

W mojej implementacji B-drzewa wierzchołki są rozszerzone o wskaźnik do rodzica, co w wielu aspektach ułatwia proces pisania kodu, jednak jednocześnie generuje potencjalne koszty. Oznacza to, że za każdym razem, kiedy przenosimy dzieci z jednego wierzchołka do drugiego, musimy również aktualizować ich wskaźniki na nowego rodzica, co potencjalnie może wymagać dodatkowych operacji dyskowych. B-drzewo wykorzystuje pliki zapisywane w formie binarnej.

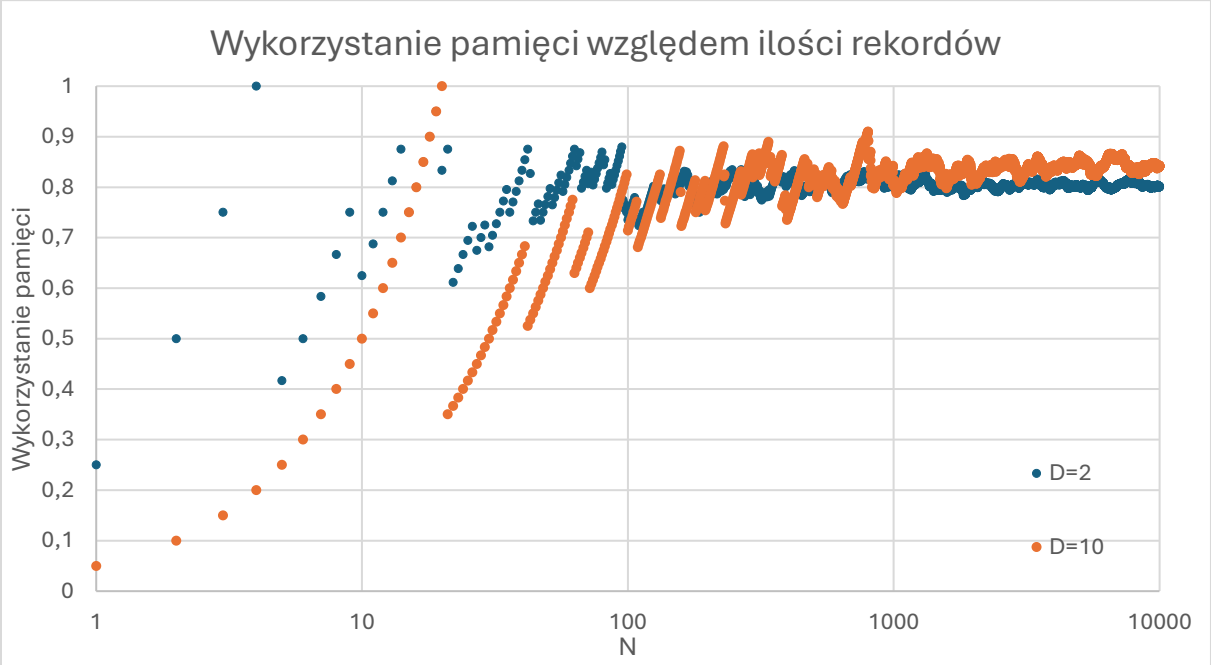
Program funkcjonuje w dwóch trybach pracy:

- Interaktywny – Umożliwia bieżącą kontrolę operacji i oglądania wizualizacji wraz z aktualnym stanem pliku z rekordami.
- Do testowania – Wczytuje serię operacji (INSERT, REMOVE, MODIFY, SEARCH) z pliku .txt i sprawdza poprawność zwracanych wyników/statusów przez funkcje B-drzewa. W tym trybie również można wykorzystać opcję wizualizacji.

Do wizualizacji zarówno struktury drzewa, jak i zawartości pliku wykorzystałem narzędzie Graphviz, które generuje pliki png.

3.2. Wykorzystanie pamięci

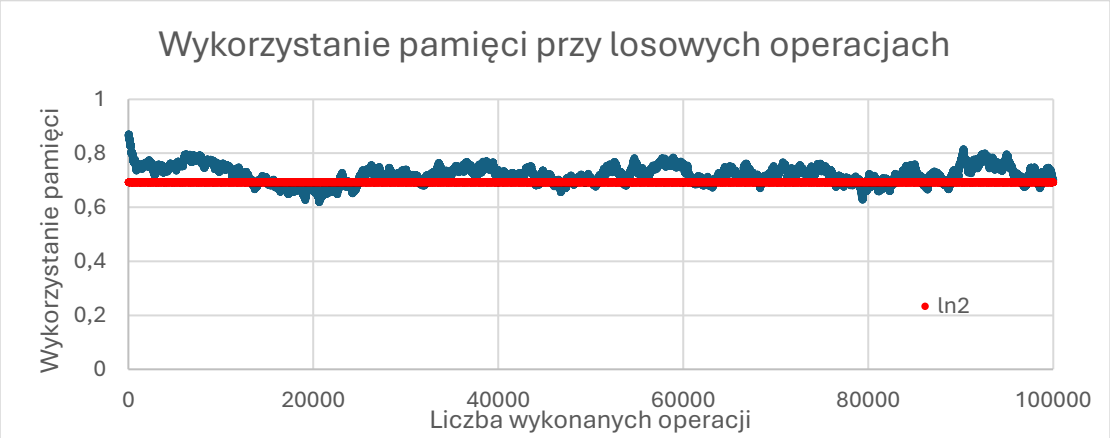
Pierwszą rzeczą jaką sprawdziłem w kontekście wykorzystania pamięci, była jej zależność od liczby rekordów. W celu zbadania tego, wykonałem $10\,000$ kolejnych operacji INSERT na początkowo pustych drzewie, sprawdzając bieżące wykorzystanie pamięci po każdej operacji. Eksperyment ten przeprowadziłem zarówno dla $d = 2$ i $d = 10$.



Jak można zauważyć, zarówno dla $d = 2$, jak i $d = 10$ widać wyraźne cykle wzrostów i spadków w wykorzystaniu pamięci. Wzrosty są wynikiem kolejnych operacji INSERT zapełniających węzły, natomiast spadki są bezpośrednim efektem operacji rozszczepienia, które muszą zostać wykonane po przepełnieniu węzła. Im mniej jest rekordów, tym takie operacje bardziej negatywnie wpływają na wahania współczynnika.

Widać, że ewidentnie z większą liczbą rekordów współczynnik ten zaczyna się stabilizować. Dla $d = 2$ osiąga wartość około 80% a $d = 10$ ok. 84%. Różnica między tymi pomiarami może wynikać z faktu, iż dla większego d stabilizacja tego współczynnika wymaga znacznie więcej rekordów.

Następny fakt, który chciałem sprawdzić, jest teoretycznie wyliczone $\ln 2$ wykorzystanie pamięci dla losowych operacji. W tym celu na drzewie o początkowej wielkości $1\ 000$, wykonałem $100\ 000$ losowych operacji (50% szans na INSERT i 50% szans na REMOVE). Następnie sprawdziłem, jak zmieniało się wykorzystanie pamięci w trakcie tych testów.



Z testów wynika, że wykorzystanie pamięci wahało się w trakcie trwania testów, ale jego wartość centralna wynosiła ok. $\mathbf{72}\%$. Jest to rzeczywiście blisko wartości teoretycznej (zaznaczonej na czerwono na wykresie), biorąc pod uwagę małą ilość rekordów użytych w trakcie eksperymentu (mogła wprowadzić niestabilność).

3.3. Ilość operacji przy wyszukiwaniu

Wiemy, że ilość operacji potrzebnych przy uzyskania klucza można przybliżyć do $\log_d N$, czyli mniej więcej do wysokości drzewa. Aby to przetestować w praktyce, dla różnych wartości d sprawdziłem, ile średnio było potrzebnych operacji przy $100\ 000$ operacjach SEARCH na drzewie z $100\ 000$ kluczami. Sprawdzałem również jaka wyniosła wysokość drzewa, by sprawdzić jej powiązanie z ilością operacji. Na potrzeby eksperymentu, nie wykorzystałem wcześniej zaimplementowanej pamięci podręcznej.

d	1	2	3	4	5	6	7	8	9	10
Średnia ilość operacji	11,37	7,69	6,80	5,85	5,88	4,90	4,91	4,92	4,93	3,94
h	12	8	7	6	6	5	5	5	5	4
h_{min}	11	8	6	6	5	5	5	5	4	4
h_{max}	16	10	8	7	7	6	6	5	5	5

Jak można zauważyć, pomierzone wartości są w pełni zgodne z oczekiwaniami. Średnia ilość operacji jest zawsze trochę mniejsza niż wysokość drzewa, a pomierzona wysokość drzewa jest zgodna z teoretycznym przedziałem, w jakim powinna się mieścić. Ciekawym zjawiskiem jest fakt, że w niektórych przypadkach zwiększenie wartości d, nie sprawiło, że ilość operacji była mniejsza. Wynika to z faktu, że mimo że wartość ta jest większa, to wysokość drzewa pozostała taka sama.

3.4. Zależność ilości operacji dyskowych od d i pamięci podręcznej

W tym eksperymencie chciałem sprawdzić, jak parametr d oraz pojemność pamięci podręcznej wpływają na ilość operacji dyskowych. W tym celu przeprowadziłem test, w którym dla drzewa z $100\,000$ kluczami wykonałem $100\,000$ operacji SEARCH i zmierzyłem sumaryczną liczbę odczytów danych z dysku. Powtórzyłem tę czynność dla różnych par d i pojemności pamięci podręcznej. Wielkość pamięci podręcznej oznaczyłem jako S i podawałem ją w liczbie stron.

S \ d	1	5	10	25	100
1	1 137 049	587 989	393 965	297 574	299 419
2	1 137 044	587 976	393 635	295 223	298 649
4	1 136 965	586 157	287 949	195 482	165 143
8	1 133 672	429 704	282 034	191 322	117 278
16	979 320	377 589	266 530	183 251	98 017

Jak widać, nawet gdy znacznie zmieniamy pojemność pamięci podręcznej, to nie widać efektów, jeżeli d jest zbyt małe. Jednak gdy zwiększamy wartość d, od razu widać spore różnice. Jednocześnie, im większe te wartości, tym poprawa staje się mniejsza.

Idealnie widać, że gdy jednocześnie obie te wartości zwiększamy, to wtedy otrzymujemy najlepsze wyniki. Porównując ilość operacji dla optymalnej pary do ilości operacji dla najgorszej pary to widać, że ta najlepsza osiągnęła aż 11 razy mniej operacji niż ta gorsza. Pokazuje to, jak istotne jest ustawienie odpowiedniej wartości d, która maksymalizuje naszą ilość pobieranych informacji, przy jednej operacji, a następnie wykorzystanie buforowania do zminimalizowania konieczności ponownego odczytu danych.

4. Podsumowanie

Przeprowadzone testy i analiza w pełni potwierdzają założenia teoretyczne B-drzew. Eksperymenty wyszukiwania kluczy potwierdziły logarytmiczną złożoność operacji na tej strukturze. Pomierzone średnie wykorzystanie pamięci przy losowych operacjach (72%) było bardzo zbliżone do teoretycznej wartości (69%). Pomiary również wykazały, że prawidłowe dostosowanie parametru d oraz zastosowanie mechanizmu buforowania są kluczowe dla optymalnego działa B-drzew. Optymalna konfiguracja poprawiła wydajność operacji ponad 10-krotnie w porównaniu do konfiguracji nieoptymalnej.